React / Next.js — Complete Freelance & Business Roadmap

v6.0 · Freelance Edition · 28+ Projects · 5 Phases + Supplements Full-time learner · 6–8 hrs/day · Gap-Sealed · Production-Ready · Business-Ready

Includes: Email Flows · File Uploads · Headless CMS · Deep SEO · Advanced Mobile UX · Git Workflow · API Design · Error Handling · WOW Factor Additions

---

How to Use This Roadmap

- Read the warm-up for each project — ask your AI to drill you on those topics first.
- Write your architecture plan before opening the code editor.
- Build until it works. Ugly and working beats pretty and broken.
- Log the project in the table at the back when done.
- Move on — do not polish endlessly.

> **Rule: The warm-up is not optional. If you skip it you will get stuck and not know why.**

Time Expectations at 6–8 hrs/day

Phase	Focus	Time per project
Phase 1	Core React	2–4 days
Phase 2	Next.js Frontend	3–5 days
Phase 3	Full Stack	5–8 days
Phase 4	Production Level	7–10 days
Phase 5	Freelance & Business	4–7 days

These are guides, not deadlines.

🔧 Supplement A — Git & Version Control

> **Do this before Project 1. Takes 1 day. Never skip it.**

WHY IT EXISTS

Every project in this roadmap requires meaningful git commits. Without understanding git properly, you will either commit everything in one blob or avoid branching entirely. This supplement fixes that before it becomes a habit.

CONCEPTS TO LEARN

- `git init`, `add`, `commit`, `push`, `pull`
- Branching: `git checkout -b feature/name`
- Merging and resolving merge conflicts
- Pull requests — what they are and why teams use them
- `.gitignore` — what to never commit (`.env`, `node_modules`)
- Conventional commits: `feat:` `fix:` `chore:` `refactor:`

- Conventional commits: `feat:`, `fix:`, `chore:`, `refactor:`
- Squashing commits before merging

PRACTICE EXERCISE

- Create a repo for your Task Manager project
- Work on a `feature/add-task` branch, open a PR to `main`, merge it
- Intentionally create a merge conflict and resolve it
- Write every commit using conventional commit format

READY WHEN

- You can branch, commit, push, open a PR, and merge without looking anything up
- You understand why `.env` must never be committed
- Every project from here uses feature branches, not commits directly to `main`

🔧 Supplement B — Error Handling Fundamentals

> **Read this before Project 3. Reference it throughout the roadmap.**

WHY IT EXISTS

The roadmap introduces error states and error boundaries across many projects but never treats error handling as a first-class skill. This supplement gives you the mental model so you stop treating errors as an afterthought.

CONCEPTS TO LEARN

- `try/catch/finally` — when and where to use it
- The difference between expected errors (user did something wrong) and unexpected errors (your code broke)
- Error boundaries in React — what they catch and what they don't
- Async error handling — why `async/await` errors don't bubble without `try/catch`
- Server action error responses — returning structured errors vs throwing
- Never swallow errors silently — always log or surface them
- The 3 layers of error boundaries in Next.js: `error.tsx`, `global-error.tsx`, component-level

PRACTICE EXERCISE

- Take your Task Manager and intentionally break it in 3 ways
- Add proper error boundaries and try/catch to recover gracefully
- Write a utility function `handleError(error)` that logs + returns a user-friendly message

READY WHEN

- You can explain the difference between a thrown error and a returned error object
- You know which errors React Error Boundaries catch and which they don't
- You never leave an `async` function without error handling

🔧 Supplement C — REST API Design Thinking

> **Read this before Phase 3. Reference it when building any API route.**

WHY IT EXISTS

The roadmap makes you consume APIs heavily and build API routes, but never teaches you to design them intentionally. Without this, you name routes by feel and return inconsistent responses.

CONCEPTS TO LEARN

- REST principles: resources, statelessness, uniform interface
- HTTP methods: `GET`, `POST`, `PUT`, `PATCH`, `DELETE` — when to use each
- Status codes that matter: `200`, `201`, `204`, `400`, `401`, `403`, `404`, `409`, `422`, `500`
- Consistent response shape: `{ data, error, message }`
- Route naming conventions: `/api/tasks` not `/api/getTasksList`
- Idempotency — which methods are idempotent and why it matters
- Versioning basics: `/api/v1/` — when and why
- The difference between authentication (who are you) and authorization (what can you do)

PRACTICE EXERCISE

- Design the full API for your Task Manager on paper before building Project 10
- For every route, write: method, path, request body, success response, error responses
- Review it after you build it — where did your design diverge from reality?

READY WHEN

- You can design a full CRUD API on paper with correct methods and status codes
- You never use `GET` to mutate data
- You return consistent response shapes across all your routes

---

Phase 1 — Core React

Thinking: Components · State · Props · Forms · Derived data · Custom hooks · TypeScript intro

Stack: Next.js App Router — add `"use client"` at the top of every component in Phase 1. TypeScript from Project 3 onwards.

---

1 · Task Manager `2 days` `P1`

NEW CONCEPTS

- `useState`, CRUD operations, derived state, basic component tree
- Controlled inputs, accessibility — `aria-label` on all buttons

REQUIREMENTS

- Add a task with a text input
- Mark a task as done (toggle) and delete a task
- Filter by: all / active / done

- Show count of remaining tasks
- Min 6 meaningful git commits

STRETCH GOAL

- Persist tasks in `localStorage` so they survive a page refresh.

⭐ WOW FACTOR (optional, do after core works)

- Add smooth animations when tasks are added/removed using CSS transitions
- Add a "Clear all completed" button with a confirmation step
- Priority levels (low/medium/high) with colour-coded badges

READY WHEN

- You can explain why `filteredTasks` is derived state and not its own `useState`
- You can explain the difference between controlled and uncontrolled inputs

---

2 · Expense Tracker `3 days` `P2`

NEW CONCEPTS

- `useReducer` — when and why
- Complex forms with multiple fields, `Object.entries`, array `reduce`
- Form labels and full keyboard accessibility

REQUIREMENTS

- Add expense: name, amount, category (4+ categories)
- Delete an expense, filter by category
- Total per category + grand total — derived, never stored in state
- Every input has a proper `label` element

STRETCH GOAL

- Build a simple bar chart using plain CSS — bar width proportional to amount.

⭐ WOW FACTOR (optional, do after core works)

- Monthly budget limits per category — show a warning when approaching the limit
- Export expenses as CSV (client-side, no library)
- Undo last delete using a "recently deleted" state buffer

READY WHEN

- You can explain why you chose `useReducer` over `useState` for this project
- You can explain the difference between a reducer, an action, and dispatch

---

2.5 · Weather Widget `2 days` `GAP-FILLER`

NEW CONCEPTS

NEW CONCEPTS

- `useEffect`, fetch API, `async/await`
- Loading states, error states, API keys in `.env.local`

REQUIREMENTS

- Search for a city by name
- Show: temperature, condition, humidity, wind speed
- Loading skeleton while fetching
- User-friendly error if city not found or fetch fails
- API key in `.env.local` — never hardcoded

STRETCH GOAL

- Add a 5-day forecast section below current weather.

⭐ **WOW FACTOR (optional, do after core works)**

- Auto-detect user location with the browser Geolocation API on first load
- Animated weather icons that change based on condition (sun, rain, clouds)
- Unit toggle: Celsius / Fahrenheit with instant re-render (no re-fetch)

READY WHEN

- You can explain what the `useEffect` dependency array controls
- You can explain the difference between a loading state and an error state

---

3 · Quiz App `3 days` `P3`

NEW CONCEPTS

- Multi-step UI state, conditional rendering, component composition
- TypeScript interfaces — starts here, no `any` allowed anywhere

REQUIREMENTS

- At least 8 hardcoded questions with 4 options each
- One question at a time with progress indicator
- Track score — results screen with score, percentage, restart
- No going back to previous questions
- Define a `Question` interface — type all component props

STRETCH GOAL

- Add a countdown timer per question — if time runs out, mark wrong and advance.

⭐ **WOW FACTOR (optional, do after core works)**

- Animated progress bar that fills as you advance through questions
- Results screen shows which questions you got wrong with the correct answer highlighted
- Confetti animation on perfect score (use `canvas-confetti` — 1 allowed library)

READY WHEN

- You can explain how you managed which screen to show using a single state variable
- You can explain what a TypeScript interface is and why it catches bugs early

---

4 · Recipe Book `3 days` `P4`

NEW CONCEPTS

- `useRef` — DOM and non-DOM uses
- Nested TypeScript types, custom hooks, TypeScript generics (basic)

REQUIREMENTS

- List recipes, click for full details: ingredients + steps
- Search bar filters by name — use `useRef`, not state
- Add a new recipe via a form
- Recipes persist in `localStorage` via a custom `useLocalStorage` hook

STRETCH GOAL

- Shopping list — select multiple recipes and combine ingredients, removing duplicates.

⭐ WOW FACTOR (optional, do after core works)

- Ingredient scaling — enter number of servings and all amounts recalculate automatically
- Print-friendly view for a recipe (CSS `@media print`)
- Import a recipe from a plain text paste — parse it into structured data with basic string manipulation

READY WHEN

- You can explain when to use `useRef` instead of `useState` for an input
- You can explain what a custom hook is and give an example of when to extract one

---

5 · Kanban Board `3 days` `P5`

NEW CONCEPTS

- `useReducer` with complex state shape
- Discriminated union actions, lifting state, immutable updates

REQUIREMENTS

- Three columns: To Do / In Progress / Done
- Add a card to any column via an inline form
- Move a card to the next column with a button
- Delete a card — show card count per column in the header

STRETCH GOAL

- Add drag and drop using the native HTML5 drag API (no library).

⭐ **WOW FACTOR (optional, do after core works)**

- Card due dates with visual overdue indicator
- WIP (work-in-progress) limits per column — warn when column exceeds limit
- Keyboard-only navigation between cards and columns (full accessibility)

READY WHEN

- You can explain why all board state lives in one place and flows down as props
- You can explain what a discriminated union is and why it makes reducers safer

---

5.5 · Theme & Settings App `2 days` `GAP-FILLER`

NEW CONCEPTS

- `useContext` + `useReducer` combined pattern — the most common scalable state pattern
- `React.createContext`, context provider, context performance

REQUIREMENTS

- Global `SettingsContext` with `useContext` + `useReducer`
- Settings: theme (light/dark), language, notifications (on/off)
- 3+ deeply nested components consuming context without prop drilling
- Split into 2 separate contexts — explain why
- TypeScript: context value fully typed, no `any`

STRETCH GOAL

- Add a reset-to-defaults action and undo/redo using a history array in the reducer.

⭐ **WOW FACTOR (optional, do after core works)**

- Respect `prefers-color-scheme` media query on first load as the default theme
- Animated theme transition (CSS variables transition smoothly instead of snapping)
- Per-component font size setting using CSS custom properties fed from context

READY WHEN

- You can build the `useContext` + `useReducer` pattern from memory
- You can explain when NOT to use context and the performance concern

---

5C · Habit Tracker `3 days` `CONSOLIDATION`

NEW CONCEPTS

- ALL Phase 1 concepts together — `useState`, `useReducer`, `useRef`, `useEffect`, `useContext`
- Custom hooks, TypeScript, derived state, `localStorage`

REQUIREMENTS

- Add/delete habits with target days per week
- Check off a habit for today — view a 7-day grid per habit
- Streak count per habit — derived, never stored
- Global settings via `useContext` + `useReducer`
- Persist everything in `localStorage` via your custom hook
- TypeScript throughout — 2+ custom hooks extracted

STRETCH GOAL

- Weekly summary screen showing completion rates as a percentage per habit.

⭐ **WOW FACTOR (optional, do after core works)**

- Longest streak ever vs current streak — both derived
- GitHub-style yearly contribution heatmap (52 weeks × 7 days grid, CSS only)
- Motivational streak milestone notifications (inline, not browser alerts)

READY WHEN

- You can build all Phase 1 patterns from memory without referencing previous projects

---

Phase 2 — Next.js Frontend

App Router · Layouts · Server components · Tailwind · Real UI patterns · Libraries

> Key shift: You now think about **where** code runs (server vs client), not just **what** it does.

---

6 · Personal Portfolio Site `3 days` `P6`

NEW CONCEPTS

- Next.js App Router, file-based routing, layouts, server vs client components
- `next/font` (prevents CLS), error boundaries (3 layers), `next.config.js`

REQUIREMENTS

- Home, About, Projects, Contact pages
- Shared Navbar and Footer via root `layout.tsx`
- Responsive design — Tailwind, works on mobile and desktop
- `not-found.tsx` and `error.tsx` boundary pages
- Deployed to Vercel — Lighthouse Accessibility score >= 90
- Fonts loaded via `next/font` only, never via a `link` tag

STRETCH GOAL

- Add a blog section using `.md` files and dynamic routes `/blog/[slug]`.

⭐ **WOW FACTOR (optional, do after core works)**

- Page transition animations using Framer Motion's `AnimatePresence`
- Interactive project cards that show a live preview iframe on hover
- Command palette (`Cmd+K`) that lets visitors jump to any section or copy your email

READY WHEN

- You can explain server vs client component and when to use each
- You can explain why `next/font` prevents CLS

---

7 · Movie / TV Show Browser `4 days` `P7`

NEW CONCEPTS

- Server-side fetch in server components, dynamic routes `[id]`
- URL search params, `loading.tsx`, React Hook Form, React Suspense & streaming

REQUIREMENTS

- Home page shows popular movies — server fetch in server component
- Detail page at `/movies/[id]` with full info
- Search bar updates results via URL search params (not just state)
- Loading skeleton while fetching — error state if API fails
- Use React Hook Form for search, Suspense to stream at least one section
- TMDB API key stored in `.env.local` — never in client components

STRETCH GOAL

- Favourites list saved to `localStorage` — note the client/server split this requires.

⭐ WOW FACTOR (optional, do after core works)

- Infinite scroll (Intersection Observer API — no library)
- Genre-based colour theming — each genre has a distinct accent colour applied via CSS variables
- "Trailer" button that opens a YouTube embed in a modal (fetch trailer key from TMDB)

READY WHEN

- You can explain server fetch vs client fetch and when to choose each
- You can explain what React Suspense does and how Next.js uses it for streaming

---

7.5 · Notes App with Rich Text `3 days` `GAP-FILLER`

NEW CONCEPTS

- Tiptap rich text editor, client vs server components in practice
- `localStorage` persistence, debouncing

REQUIREMENTS

- Create, edit, and delete notes with title and rich text body (Tiptap)

- Create, edit, and delete notes with title and rich text body (Tiptap)
- Toolbar: bold, italic, heading, bullet list
- Notes list in sidebar, selected note in main area
- Auto-save to `localStorage` as user types — debounced, not on every keystroke
- Search/filter notes by title

STRETCH GOAL

- Add tags to notes and filter by tag.

⭐ WOW FACTOR (optional, do after core works)

- Word count and estimated read time shown live as you type
- Markdown export — convert Tiptap JSON to `.md` and trigger a file download
- Pinned notes that always appear at the top regardless of sort order

READY WHEN

- You can explain why a rich text editor requires `use client`
- You understand debouncing and why you would not save on every keystroke

---

8 · Dashboard UI `4 days` `P8`

NEW CONCEPTS

- Complex layouts, Recharts (`LineChart` + `BarChart`)
- Sortable/filterable table, Zod, `useMemo`, `useCallback`, `useTransition`

REQUIREMENTS

- Sidebar navigation — collapsible on mobile
- 4+ stats cards: users, revenue, active sessions, churn rate
- At least one line chart and one bar chart using Recharts
- Data table with sort by column and text search filter
- Zod schema for all data shapes — `z.array(schema).parse(data)` at startup
- `useMemo` on sort/filter computation — `useTransition` on table search

STRETCH GOAL

- Date range picker that filters both charts and table simultaneously.

⭐ WOW FACTOR (optional, do after core works)

- Animated number counters on the stats cards (count up from 0 on mount)
- CSV export of the filtered table data (client-side, no library)
- Persistent column visibility toggles — user can hide/show columns, saved to `localStorage`

READY WHEN

- You can explain the difference between `useMemo` and `useCallback`
- You can explain what `useTransition` does and give a concrete example

---

9 · E-commerce Store (Frontend) `5 days` `P9`

NEW CONCEPTS

- React Context for global cart state, URL-based filter state
- React Hook Form + Zod full validation, derived cart totals

REQUIREMENTS

- Product listing with filter by category and price range
- Product detail page — add to cart, remove, update quantity
- Cart total is derived — never stored in state
- Cart in React Context so navbar shows item count
- Active filters reflected in the URL — page is shareable
- Checkout form: React Hook Form + Zod — inline validation errors

STRETCH GOAL

- Product comparison — select up to 3 products and compare specs side by side.

⭐ WOW FACTOR (optional, do after core works)

- Recently viewed products (stored in `localStorage`, shown in a horizontal scroll strip)
- Stock level indicator with urgency copy ("Only 3 left")
- Cart drawer that slides in from the right (CSS transforms, no library)

READY WHEN

- You can explain why the cart lives in Context and not in a component's `useState`
- You can explain the difference between Zod schema validation and manual if/else

---

9C · Music / Podcast Library `4 days` `CONSOLIDATION`

NEW CONCEPTS

- ALL Phase 2 concepts — App Router, server fetch, dynamic routes, Context
- React Hook Form + Zod, Recharts, Tailwind layouts

REQUIREMENTS

- Browse albums/podcasts from iTunes Search API (free, no key)
- Dynamic route for each item: `/albums/[id]`
- Search bar with URL search params
- Favourites list in Context + `localStorage`
- Listening stats page with at least one Recharts chart
- Submit a recommendation form with React Hook Form + Zod

STRETCH GOAL

- Playlist feature — drag to reorder tracks (HTML5 drag API).

⭐ **WOW FACTOR (optional, do after core works)**

- Audio preview player — play the 30-second iTunes preview clip inline
- Queue system — add multiple tracks to a play queue that persists across navigation
- Animated equaliser bars on the currently playing track thumbnail

READY WHEN

- You can build all Phase 2 patterns from memory without referencing previous projects

---

Phase 3 — Full Stack

Prisma · PostgreSQL · NextAuth · Server Actions · TanStack Query · Role-based access

> Key shift: You now think about the **full request lifecycle** and **data ownership**. Every operation must be verified server-side.

---

10 · Full Stack Task Manager `5 days` `P10`

NEW CONCEPTS

- Prisma ORM, PostgreSQL, Next.js API routes + server actions
- NextAuth.js sessions, TanStack Query, `useActionState` (React 19), `server-only` package

REQUIREMENTS

- Sign up and log in — NextAuth with GitHub OAuth or email/password
- Tasks saved to PostgreSQL via Prisma — each user sees ONLY their own tasks
- Protected routes — unauthenticated users redirected to `/login`
- TanStack Query for all client-side data fetching
- Server action responses handled with `useActionState` — errors shown inline
- `prisma/seed.ts` with test data — `server-only` on all DB lib files

STRETCH GOAL

- Add due dates and email reminders using Resend (free tier).

⭐ **WOW FACTOR (optional, do after core works)**

- Optimistic updates with TanStack Query — task toggles feel instant before server confirms
- Activity log — every task change is stored in a separate `TaskEvent` table and shown as a timeline
- Public shareable task list — generate a read-only URL for a task list that anyone can view

READY WHEN

- You understand the full request lifecycle: browser → server → DB → response
- You can explain `prisma migrate dev` vs `prisma migrate deploy`
- You can explain what `server-only` does and why it catches secret leaks at build time

---

10.5 · Link Shortener `2 days` `GAP-FILLER`

NEW CONCEPTS

- Server actions with `redirect()`, `prisma.create/findUnique/upsert`
- `nanoid` for slug generation, rate limiting with Upstash

REQUIREMENTS

- Paste a long URL and get a short link (e.g. `yourapp.com/abc123`)
- Visiting the short link redirects to the original URL
- Track click count per link — increment on each visit
- User must be logged in to create links
- Rate limiting: max 10 links created per user per hour

STRETCH GOAL

- Let the user choose a custom slug instead of auto-generating.

⭐ WOW FACTOR (optional, do after core works)

- Click analytics — track referrer, country (from request headers), and time of click
- QR code generation for each short link (use `qrcode` npm package)
- Link expiry — user sets an expiry date, expired links return a friendly page

READY WHEN

- You can write a server action that reads a form, writes to DB, and redirects
- You can explain why rate limiting is server-side and how to implement it

---

11 · Blog with CMS `6 days` `P11`

NEW CONCEPTS

- Role-based access (admin vs public), Tiptap rich text
- Image uploads to Cloudinary/Vercel Blob, Sentry error monitoring

REQUIREMENTS

- Public: browse published posts, read at `/blog/[slug]`
- Admin: log in, access `/admin`, create/edit/publish/delete posts
- Posts: title, rich text (Tiptap), cover image, slug, draft/published status
- Images uploaded to Cloudinary or Vercel Blob
- Role enforced server-side in middleware AND in every server action
- Sentry installed and capturing errors in production

STRETCH GOAL

- Add comments — authenticated users comment, admin can delete any comment.

⭐ WOW FACTOR (optional, do after core works)

⭐ **WOW FACTOR (optional, do after core works)**

- Reading time estimate on each post (derived from word count)
- Table of contents auto-generated from Tiptap headings, with scroll-spy active state
- Post series — link related posts in order with prev/next navigation

READY WHEN

- You can explain how role-based access is enforced at the server level, not just UI
- You can explain why you check the role in the server action even though the route is protected

---

12 · SaaS Starter — Team Project Manager `7 days` `P12`

NEW CONCEPTS

- Multi-tenancy, workspace/member data model, invitation flow
- Prisma relations at scale, Stripe billing, webhook signature verification + idempotency

REQUIREMENTS

- Sign up and create a workspace — invite users by email
- Create projects and tasks inside a workspace
- Members see ONLY workspaces they belong to — roles: owner / member
- All data scoped to workspace — no cross-workspace leaks
- Free plan: max 3 projects per workspace — enforced server-side
- Stripe Checkout for upgrading to paid — handle `checkout.session.completed` webhook
- Webhook signature verified — processed event IDs stored in DB (idempotency)

STRETCH GOAL

- Add a Stripe customer portal so users can manage billing and cancel subscriptions.

⭐ **WOW FACTOR (optional, do after core works)**

- Workspace audit log — every significant action (invite sent, project created, plan changed) is logged with actor + timestamp
- Seat-based billing — price scales with number of members, enforced in Stripe and server-side
- In-app notifications — bell icon with unread count, powered by a `Notification` table in DB

READY WHEN

- You can explain how you scoped every database query to a workspace
- You can explain the Stripe payment flow end to end
- You can explain why webhook idempotency matters and how you implemented it

---

12C · Real-time Chat App `5 days` `CONSOLIDATION`

NEW CONCEPTS

- Pusher Channels for real-time, `useOptimistic` (React 19)
- `useOptimistic` + Server Action pattern — all Phase 3 patterns consolidated

REQUIREMENTS

- Auth with NextAuth — create and join chat rooms
- Send and receive messages in real time using Pusher (free tier)
- Messages persisted to PostgreSQL — history loads on join
- Show who is online in a room (presence channel)
- Messages appear instantly using `useOptimistic` before server confirms
- At least one mutation: `useOptimistic` + Server Action directly (not TanStack Query)

STRETCH GOAL

- Add message reactions (emoji) — real-time updates for all room members.

⭐ WOW FACTOR (optional, do after core works)

- Typing indicators — "Alice is typing…" shown in real time via Pusher presence events
- Message threading — reply to a specific message, shown as an indented thread
- Giphy integration — `/gif search-term` command inserts a GIF inline

READY WHEN

- You can build all Phase 3 patterns from memory without referencing previous projects

---

Phase 4 — Production Level

Testing · CI/CD · Performance · Accessibility · Monitoring · Ship it

> Key shift: You now think about **reliability**, **maintainability**, **monitoring**, and what happens when real users break things.

---

13 · Add Full Test Coverage to a Previous Project `7 days` `P13`

NEW CONCEPTS

- Vitest, React Testing Library, Playwright e2e
- GitHub Actions CI, MSW (Mock Service Worker), `vi.mock` for Next.js internals

REQUIREMENTS

- Unit tests for all utility/helper functions and all `useReducer` reducers
- Component tests: form submit, add item, delete item, filter
- At least one MSW mock for an API call — test both success and error paths
- `next/navigation` mocked with `vi.mock` in all component tests that use routing
- At least one Playwright e2e test: full login flow
- GitHub Actions CI pipeline — tests run on every push, PR blocked if tests fail
- Test coverage report generated (`vitest --coverage`)

STRETCH GOAL

- Write tests before writing the component (TDD) for at least one new feature.

⭐ WOW FACTOR (optional, do after core works)

- Visual regression tests using Playwright screenshots — catch unintended UI changes
- Test the optimistic update path — assert the UI updates before the server responds
- Coverage badge in your README pulled from your CI pipeline

READY WHEN

- You can write a component test before writing the component itself
- You can explain unit vs integration vs end-to-end test
- You can mock an API call using MSW and test both success and error paths

---

13.5 · Performance & Accessibility Audit `3 days` `GAP-FILLER`

NEW CONCEPTS

- Lighthouse, Core Web Vitals (LCP/CLS/INP), WCAG 2.1 AA
- `next/image` advanced props, bundle analysis, Sentry performance, feature flags

REQUIREMENTS

- Pick any Phase 2 or 3 project — document scores before and after
- Achieve Lighthouse Performance score >= 90 on desktop
- Fix all critical axe accessibility errors (0 critical errors)
- Achieve Lighthouse Accessibility score >= 90
- Add skip-to-content link — all interactive elements have visible focus indicators
- `next/image` with correct `sizes`, `priority`, and `placeholder` props
- Sentry performance tracing — identify and fix at least one slow DB query
- Implement one feature behind a feature flag
- Write a 1-page audit report: what you found, what you fixed, scores before/after

STRETCH GOAL

- Achieve a perfect 100 Lighthouse score across all 4 categories.

⭐ WOW FACTOR (optional, do after core works)

- Set up `@next/bundle-analyzer` and eliminate at least one large dependency
- Add `web-vitals` library and send real user metrics to a logging endpoint
- Document your performance findings as a public case study on your portfolio

READY WHEN

- You can run a Lighthouse audit and understand every metric
- You can identify and fix common accessibility violations
- You can explain the `sizes`, `priority`, and `placeholder` props on `next/image`

---

14 · Your Own Real Product `Your timeline` `P14`

NEW CONCEPTS

- Product thinking, real users, every skill from the roadmap

REQUIREMENTS

- Deployed and publicly accessible (Vercel is fine)
- Real users — even just 5 friends or family members using it counts
- At least one paid feature or a waiting list (Stripe or Lemon Squeezy)
- Sentry installed and capturing production errors
- Rate limiting on all mutation endpoints
- `README.md`: what it does, how to run locally, architecture decisions

STRETCH GOAL

- Write a public case study — publish on your portfolio.

⭐ WOW FACTOR (non-optional for this project — this is your showpiece)

- Changelog page — every significant update logged publicly (`/changelog`)
- Status page — uptime indicator using a simple health check endpoint
- Onboarding flow — new users are guided through key features on first login

READY WHEN

- This is your portfolio centrepiece — there is no next step except to keep building.

Phase 5 — Freelance & Business Skills

Email Flows · File Uploads · Headless CMS · Deep SEO · Advanced Mobile · Business Launch

> Why Phase 5 exists: The original roadmap makes you production-ready. Phase 5 makes you **freelance-ready** and **business-ready**. These are the skills that come up in real client work and when running your own product.

P5-A · Transactional Email System `4 days` `NEW`

NEW CONCEPTS

- React Email — building email templates in JSX
- Resend API, welcome email on signup, password reset flow (token + expiry)

REQUIREMENTS

- Welcome email sent automatically on user registration
- Password reset: request form → email with signed token → reset form → token validated server-side
- Email template built with React Email (not raw HTML strings)

- Tokens expire after 1 hour — stored in DB
- Test email preview locally with React Email dev server

STRETCH GOAL

- Order confirmation email for your e-commerce project — HTML receipt with line items.

⭐ WOW FACTOR (optional, do after core works)

- Weekly digest email — summarise user activity from the past 7 days, sent via a cron job (Vercel Cron)
- Email open tracking — pixel-based, store `openedAt` timestamp on the token record
- Unsubscribe link in every email — one-click, verified server-side, stored in DB

READY WHEN

- You can build a full password reset flow from scratch
- You can build email templates in JSX with React Email

P5-B · Advanced File Uploads `4 days` `NEW`

NEW CONCEPTS

- Presigned URLs — browser uploads directly to S3/Cloudflare R2 (no server proxy)
- UploadThing, file validation server-side, image optimisation (sharp), bulk upload

REQUIREMENTS

- Single image upload with preview and crop (`react-image-crop`)
- Bulk upload: multiple files, progress bar per file, cancel support
- Presigned URL flow: client requests URL → uploads direct to storage → server saves URL to DB
- File type whitelist + max size enforced server-side — not just client
- Image resized to multiple sizes on upload using `sharp`
- Video upload with size warning and progress indicator

STRETCH GOAL

- Drag-and-drop upload zone with file reordering before submit.

⭐ WOW FACTOR (optional, do after core works)

- Blurhash placeholder generated server-side on upload — shown while the real image loads
- Storage usage indicator — show user how much of their quota they've used
- Background upload — user can navigate away and upload continues in the background

READY WHEN

- You can explain the presigned URL flow and why it avoids proxying through your server
- You can validate files server-side and explain why client-only validation is insufficient

---

P5-C · Headless CMS Integration — Sanity.io `5 days` `NEW`

NEW CONCEPTS

- Sanity Studio — content modelling, schemas, GROQ query language
- Portable Text rendering, ISR + webhook revalidation, draft/preview mode

REQUIREMENTS

- Set up Sanity project and define schemas: Post, Author, Category
- Build a blog — editors write in Sanity Studio, readers visit Next.js
- Fetch posts with GROQ and render with `next-sanity` + Portable Text
- Dynamic routes `/blog/[slug]` — slug comes from Sanity
- ISR: page revalidates when content is published (Sanity webhook → `revalidatePath`)
- Draft mode — editors preview unpublished content on the live site

STRETCH GOAL

- Add a page builder — editors assemble pages from modular content blocks in Sanity.

⭐ WOW FACTOR (optional, do after core works)

- Live preview in Sanity Studio — changes appear in the Next.js preview pane in real time
- Related posts — GROQ query that finds posts with overlapping categories
- Estimated reading time derived from Portable Text word count, shown on the post card

READY WHEN

- You can model content in Sanity and query it with GROQ
- You understand ISR + webhook revalidation flow end to end

P5-D · Deep SEO Implementation `3 days` `NEW`

NEW CONCEPTS

- `generateMetadata` per page, dynamic OG images with `next/og` (`ImageResponse`)
- JSON-LD structured data, XML sitemap, `robots.txt`, canonical URLs

REQUIREMENTS

- Add `generateMetadata` to every page — unique title, description, OG tags
- Dynamic OG image for blog posts using `next/og` — title and author rendered as image
- JSON-LD on: homepage (`WebSite`), blog posts (`Article`), product pages (`Product`)
- Auto-generated `sitemap.xml` — updates when content changes
- `robots.txt` allowing all crawlers, blocking `/admin` and `/api`
- Canonical tag on all pages — verify in Google Search Console
- Lighthouse SEO score >= 95 on all key pages

STRETCH GOAL

- Implement hreflang tags for multi-language support.

⭐ **WOW FACTOR (optional, do after core works)**

- Breadcrumb JSON-LD on all nested pages — improves Google rich results
- Auto-generated OG images for every page type, not just blog posts
- Google Search Console integration — verify indexing and fix any crawl errors live

READY WHEN

- You can add complete SEO to any Next.js page from memory
- You can explain the difference between OG tags and JSON-LD and what each is for
- You can generate dynamic OG images with `next/og`

P5-E · Advanced Responsive & Mobile UX `3 days` `NEW`

NEW CONCEPTS

- Container queries, mobile navigation patterns (bottom tab bar, drawer)
- Touch gestures (swipe, pull-to-refresh), fluid typography with `clamp()`, safe area insets

REQUIREMENTS

- Rebuild portfolio nav as a mobile bottom tab bar on small screens
- Add swipe-to-delete on a list — at least one component uses touch gestures
- Implement pull-to-refresh on a data list
- All font sizes use fluid typography with `clamp()` — no abrupt jumps at breakpoints
- Safe area padding on all full-screen layouts
- No horizontal scroll on any mobile viewport — Lighthouse Mobile Performance >= 85

STRETCH GOAL

- Native-app-style page transition (slide left/right) using Framer Motion.

⭐ **WOW FACTOR (optional, do after core works)**

- Haptic feedback on key actions using the Vibration API (supported on Android)
- Install prompt for PWA — add `manifest.json` and a service worker so the app is installable
- Pinch-to-zoom on image galleries using touch event coordinates (no library)

READY WHEN

- You can explain when to use container queries instead of media queries
- Your apps feel native on mobile, not just a small desktop

14.5 · Business Launch Kit `5 days` `NEW`

NEW CONCEPTS

- Landing page structure, waitlist with email capture
- Privacy-friendly analytics (Plausible/Umami), cookie consent, GDPR basics, legal pages

REQUIREMENTS

- Build a production landing page for your Project 14 product
- Hero, features, social proof, pricing, FAQ, CTA sections
- Email waitlist capture — stored in DB + welcome email sent via Resend
- OG image and JSON-LD structured data
- Privacy-friendly analytics — Plausible free tier or self-hosted Umami
- Cookie consent banner (if using any tracking)
- Auto-generated sitemap — Privacy Policy and Terms of Service pages
- Google Search Console verified
- Mobile score >= 85, SEO score >= 95 in Lighthouse

STRETCH GOAL

- A/B test two headline variants — track which converts better using your analytics tool.

⭐ WOW FACTOR (optional, do after core works)

- Social proof ticker — live or simulated recent signups ("John from London just signed up")
- Loom video embed in the hero section — product demo under 60 seconds
- Referral system — users get a unique invite link, tracked in DB, shown on their dashboard

READY WHEN

- You can launch a production-ready marketing site that ranks on Google
- You have a real product with a real landing page that real people can find

---

Complete Technology Checklist

Use before job applications and freelance pitches to verify you have touched every key technology.

React Core

- ☐ `useState`, `useReducer`, `useRef`, `useEffect`
- ☐ `useActionState` (React 19)
- ☐ `useMemo`, `useCallback`
- ☐ Component composition, props, derived state
- ☐ `useTransition` / `startTransition`
- ☐ Controlled forms
- ☐ `useContext`, custom hooks
- ☐ Error boundaries (3 layers)
- ☐ `useContext` + `useReducer` combined pattern
- ☐ React Suspense and streaming
- ☐ `useOptimistic` (React 19)

TypeScript

- ☐ Interfaces and type aliases
- ☐ Discriminated unions
- ☐ Generics (basic)

- [ ] Generics (basic)
- [ ] Typing props, state, reducers, context values

Next.js

- [ ] App Router + file-based routing
- [ ] `next/link`, `next/font`
- [ ] Server vs client components
- [ ] URL search params
- [ ] Dynamic routes `[id]`
- [ ] Environment variables (scoping)
- [ ] Layouts, `loading.tsx`, `error.tsx`, `not-found.tsx`
- [ ] `next.config.js` (image domains, redirects)
- [ ] Server actions + API routes
- [ ] Streaming with Suspense
- [ ] `next/image` (sizes, priority, placeholder)
- [ ] `generateMetadata` + dynamic OG images

Data & Forms

- [ ] React Hook Form
- [ ] PostgreSQL basics
- [ ] Zod validation
- [ ] `prisma migrate dev` vs `prisma migrate deploy`
- [ ] TanStack Query (`useQuery`, `useMutation`, optimistic updates)
- [ ] Database seeding (`prisma/seed.ts`)
- [ ] Prisma ORM
- [ ] Debouncing

Auth & Security

- [ ] NextAuth.js sessions
- [ ] Environment secret management
- [ ] Protected routes + middleware
- [ ] `server-only` package
- [ ] Server-side session verification
- [ ] Rate limiting (Upstash or equivalent)

- [ ] Role-based access control

Billing & Payments

- [ ] Stripe Checkout session
- [ ] Webhook idempotency (deduplicating replayed events)
- [ ] Stripe webhook handler
- [ ] Plan enforcement server-side
- [ ] Webhook signature verification
- [ ] Stripe customer portal (stretch)

Git & Collaboration *(Supplement A)*

- [ ] Feature branch workflow
- [ ] Conventional commits (`feat:`, `fix:`, `chore:`, `refactor:`)
- [ ] Pull requests and code review process
- [ ] Merge conflict resolution
- [ ] `.gitignore` — what never to commit

- [ ] Squashing commits

Error Handling *(Supplement B)*

- [ ] `try/catch/finally` in async functions
- [ ] React Error Boundaries — 3 layers in Next.js
- [ ] Structured server action error responses
- [ ] Never swallowing errors silently
- [ ] User-friendly error messages vs developer error logs

API Design *(Supplement C)*

- [ ] REST resource naming conventions
- [ ] Correct HTTP method usage (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`)
- [ ] HTTP status codes (200, 201, 204, 400, 401, 403, 404, 409, 422, 500)
- [ ] Consistent response shape `{ data, error, message }`
- [ ] Idempotency understanding
- [ ] Authentication vs authorisation distinction

Email *(Phase 5)*

- [ ] React Email templates
- [ ] Password reset flow (token + expiry)
- [ ] Resend API integration
- [ ] Order / transactional notifications
- [ ] Welcome email on signup

File Uploads *(Phase 5)*

- [ ] Presigned URL flow (direct browser to storage)
- [ ] Image resizing with `sharp`
- [ ] UploadThing integration
- [ ] Bulk upload with per-file progress
- [ ] File type + size validation (server-side)
- [ ] Video upload handling

Headless CMS *(Phase 5)*

- [ ] Sanity content modelling + schemas
- [ ] ISR + webhook revalidation
- [ ] GROQ query language
- [ ] Draft / preview mode
- [ ] Portable Text rendering

SEO *(Phase 5)*

- [ ] `generateMetadata` per page
- [ ] `robots.txt` configuration
- [ ] Dynamic OG images (`next/og`)
- [ ] Canonical URLs
- [ ] JSON-LD structured data
- [ ] Google Search Console verified
- [ ] XML sitemap generation

Mobile & Responsive *(Phase 5)*

- [ ] Container queries

- ☐ Fluid typography with `clamp()`
- ☐ Mobile navigation patterns
- ☐ Safe area insets
- ☐ Touch gestures (swipe, pull-to-refresh)
- ☐ Mobile Lighthouse >= 85

Testing & Production

- ☐ Vitest unit tests
- ☐ Vercel deployment
- ☐ React Testing Library
- ☐ Lighthouse / Core Web Vitals
- ☐ MSW (Mock Service Worker)
- ☐ Accessibility (WCAG / axe)
- ☐ `vi.mock` for module mocking
- ☐ Git workflow + meaningful commits
- ☐ Playwright e2e tests
- ☐ Bundle analysis (`@next/bundle-analyzer`)
- ☐ GitHub Actions CI

Monitoring & Observability

- ☐ Sentry error monitoring
- ☐ Feature flags via environment variables
- ☐ Sentry performance tracing
- ☐ Structured error responses from server actions
- ☐ Privacy-friendly analytics (Plausible/Umami)

Project Log

Fill this in as you complete each project.

#	Project	Started	Finished	Days	Key Lesson Learned
S-A	Git & Version Control				
1	Task Manager				
2	Expense Tracker				
2.5	Weather Widget ◆				
S-B	Error Handling				
3	Quiz App				
4	Recipe Book				
5	Kanban Board				
5.5	Theme & Settings App ◆				
5C	Habit Tracker ■				
S-C	REST API Design				
6	Portfolio Site				

7	Movie Browser
7.5	Notes App ◆
8	Dashboard UI
9	E-commerce Store
9C	Music Library ■
10	Full Stack Tasks
10.5	Link Shortener ◆
11	Blog CMS
12	SaaS Starter
12C	Chat App ■
13	Test Coverage
13.5	Perf & A11y Audit ◆
14	Your Own Product
P5-A	Email System ★
P5-B	File Uploads ★
P5-C	Headless CMS ★
P5-D	Deep SEO ★
P5-E	Advanced Responsive ★
14.5	Business Launch Kit ★

◆ Gap-filler ■ Consolidation project ★ Phase 5 / Freelance & Business

---

v6.0 — Additions over v5.0: Supplement A (Git) · Supplement B (Error Handling) · Supplement C (REST API Design) · WOW FACTOR additions for all 28 projects · Expanded technology checklist · Updated project log